

Data Communication & Networks Lab Manual

Complete 10 Practical Experiments with Sockets, Bit Stuffing, CRC, Dijkstra & Viva-Voce Bank

Author: Shaswat Raj (BIT Mesra CSE) Academic Scheme: NEP 2024–25 (5th Sem) Credits: 1.5 Practical Credits

Experiment 1: Network Configuration & Packet Analysis with Linux CLI & Wireshark

Objective: Master Linux/UNIX network administration utilities (`ip`, `ifconfig`, `ping`, `traceroute`, `netstat`, `arp`, `nslookup`, `ss`, `tcpdump`) and capture TCP 3-way handshake packets via Wireshark.

Core CLI Commands & Network Diagnostics

```
# 1. Inspect IP address and network interface link status
ip -c addr show
ip route show

# 2. Test ICMP end-to-end latency and packet loss
ping -c 4 8.8.8.8

# 3. Trace L3 routing hops via UDP/ICMP TTL expiration
traceroute bitmesra.ac.in

# 4. Inspect active TCP listening sockets and open ports
ss -tuln
netstat -rn

# 5. Capture live HTTP/DNS packet streams with tcpdump
sudo tcpdump -i any -n -c 10 'port 53 or port 80'
```

Wireshark TCP 3-Way Handshake Packet Capture Analysis

- **Packet 1 (SYN):** Client → Server (`[SYN] Seq=0, Win=65535, MSS=1460`).
- **Packet 2 (SYN-ACK):** Server → Client (`[SYN, ACK] Seq=0, Ack=1, Win=65535, MSS=1460`).
- **Packet 3 (ACK):** Client → Server (`[ACK] Seq=1, Ack=1, Win=65535`). *Connection Established!*

Experiment 2: Character (Byte) Stuffing & Destuffing Algorithm in C

Objective: Implement Data Link Layer framing using character stuffing with flag bytes (`'DLE'`, `'STX'`, `'ETX'`). Any occurrence of `'DLE'` inside the payload is escaped by inserting an extra `'DLE'`.

```

#include <stdio.h>
#include <string.h>

#define DLE 16
#define STX 2
#define ETX 3

void character_stuff(const char *input, char *stuffed) {
    int j = 0;
    stuffed[j++] = DLE;
    stuffed[j++] = STX;

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == 'D' && input[i+1] == 'L' && input[i+2] == 'E') {
            stuffed[j++] = 'D'; stuffed[j++] = 'L'; stuffed[j++] = 'E'; // Stuff extra DLE
        }
        stuffed[j++] = input[i];
    }
    stuffed[j++] = DLE;
    stuffed[j++] = ETX;
    stuffed[j] = '\0';
}

void character_destuff(const char *stuffed, char *destuffed) {
    int j = 0;
    int len = strlen(stuffed);
    // Strip starting DLE STX (2 bytes) and ending DLE ETX (2 bytes)
    for (int i = 2; i < len - 2; i++) {
        if (stuffed[i] == 'D' && stuffed[i+1] == 'L' && stuffed[i+2] == 'E' &&
            stuffed[i+3] == 'D' && stuffed[i+4] == 'L' && stuffed[i+5] == 'E') {
            i += 3; // Skip stuffed DLE
        }
        destuffed[j++] = stuffed[i];
    }
    destuffed[j] = '\0';
}

int main() {
    char data[] = "BIT_MESRA_DLE_CSE_5TH_SEM";
    char stuffed[256], destuffed[256];
    printf("Original Data Payload:  %s\n", data);
    character_stuff(data, stuffed);
    printf("Transmitted Frame Stream: DLE STX %s DLE ETX\n", stuffed);
    character_destuff(stuffed, destuffed);
    printf("Receiver Destuffed Data: %s\n", destuffed);
    return 0;
}

```

Experiment 3: Bit Stuffing & Destuffing Algorithm in C (HDLC Standard)

Objective: Implement the HDLC Bit Stuffing algorithm in C. Delimiter flag is `01111110`. The sender automatically inserts a `0` bit after every five consecutive `1`'s in the payload stream.

```

#include <stdio.h>
#include <string.h>

void bit_stuff(const char *in, char *out) {
    int count = 0, j = 0;
    for (int i = 0; in[i] != '\0'; i++) {
        out[j++] = in[i];
        if (in[i] == '1') {
            count++;
            if (count == 5) {
                out[j++] = '0'; // Stuff 0
                count = 0;
            }
        } else {
            count = 0;
        }
    }
    out[j] = '\0';
}

void bit_destuff(const char *in, char *out) {
    int count = 0, j = 0;
    for (int i = 0; in[i] != '\0'; i++) {
        out[j++] = in[i];
        if (in[i] == '1') {
            count++;
            if (count == 5) {
                i++; // Skip the stuffed 0 bit!
                count = 0;
            }
        } else {
            count = 0;
        }
    }
    out[j] = '\0';
}

int main() {
    char bits[] = "011111101111101011111110";
    char stuffed[128], destuffed[128];
    printf("Original Bit Stream:  %s\n", bits);
    bit_stuff(bits, stuffed);
    printf("Stuffed Transmitted:  01111110 %s 01111110\n", stuffed);
    bit_destuff(stuffed, destuffed);
    printf("Destuffed at Receiver: %s\n", destuffed);
    return 0;
}

```

Experiment 4: Hamming (7,4) Single-Bit Error Detection & Correction in C

Objective: Implement the Hamming (7,4) code. Generate parity bits P_1, P_2, P_4 , simulate channel bit corruption, calculate syndrome vector $S = S_2S_1S_0$, and autonomously correct the damaged bit.

Hamming (7,4) Code Matrix Equations

- $$P_1 = D_3 \oplus D_5 \oplus D_7 \quad P_2 = D_3 \oplus D_6 \oplus D_7 \quad P_4 = D_5 \oplus D_6 \oplus D_7$$
- $$\text{Syndrome Vector: } S_1 = R_1 \oplus R_3 \oplus R_5 \oplus R_7, S_2 = R_2 \oplus R_3 \oplus R_6 \oplus R_7, S_4 = R_4 \oplus R_5 \oplus R_6 \oplus R_7$$
- $$\text{Error Position } \mathbf{E} = S_4 \cdot 4 + S_2 \cdot 2 + S_1 \cdot 1$$

```

#include <stdio.h>

int main() {
    int data[4]; // 4 data bits: D3, D5, D6, D7
    int code[8]; // 7-bit codeword: index 1 to 7
    printf("Enter 4 data bits (e.g. 1 0 1 1): ");
    scanf("%d %d %d %d", &data[0], &data[1], &data[2], &data[3]);

    code[3] = data[0]; code[5] = data[1]; code[6] = data[2]; code[7] = data[3];
    code[1] = code[3] ^ code[5] ^ code[7]; // P1
    code[2] = code[3] ^ code[6] ^ code[7]; // P2
    code[4] = code[5] ^ code[6] ^ code[7]; // P4

    printf("Transmitted Codeword: ");
    for (int i = 1; i ≤ 7; i++) printf("%d", code[i]);
    printf("\n");

    // Simulate single bit channel error
    printf("Enter bit position to corrupt (1-7, or 0 for none): ");
    int corrupt_pos;
    scanf("%d", &corrupt_pos);
    if (corrupt_pos ≥ 1 && corrupt_pos ≤ 7) code[corrupt_pos] ^= 1;

    // Receiver Syndrome Calculation
    int s1 = code[1] ^ code[3] ^ code[5] ^ code[7];
    int s2 = code[2] ^ code[3] ^ code[6] ^ code[7];
    int s4 = code[4] ^ code[5] ^ code[6] ^ code[7];
    int error_pos = s4 * 4 + s2 * 2 + s1 * 1;

    if (error_pos == 0) {
        printf("✅ No error detected in transmission!\n");
    } else {
        printf("❌ Error detected at Bit Position: %d\n", error_pos);
        code[error_pos] ^= 1; // Auto-correct bit
        printf("🔧 Corrected Codeword: ");
        for (int i = 1; i ≤ 7; i++) printf("%d", code[i]);
        printf("\n");
    }
    return 0;
}

```

Experiment 5: Cyclic Redundancy Check (CRC-16 & CRC-32) in C

Objective: Implement polynomial division modulo-2 arithmetic for CRC calculation in C. Append CRC remainder at sender, inject burst errors, and verify receiver syndrome.

```

#include <stdio.h>
#include <string.h>

void xor_op(char *dividend, const char *divisor) {
    for (int i = 1; i < strlen(divisor); i++) {
        dividend[i] = (dividend[i] == divisor[i]) ? '0' : '1';
    }
}

void compute_crc(char *data, const char *gen, char *rem) {
    int data_len = strlen(data);
    int gen_len = strlen(gen);
    char temp[128];
    strcpy(temp, data);
    for (int i = 0; i < gen_len - 1; i++) strcat(temp, "0");

    for (int i = 0; i < data_len; i++) {
        if (temp[i] == '1') {
            xor_op(&temp[i], gen);
        }
    }
    strncpy(rem, &temp[data_len], gen_len - 1);
    rem[gen_len - 1] = '\0';
}

int main() {
    char data[64] = "11010011101100";
    char gen[16] = "1011"; // Generator Polynomial: x^3 + x + 1
    char rem[16];

    compute_crc(data, gen, rem);
    printf("Data Payload:      %s\n", data);
    printf("CRC Generator:      %s\n", gen);
    printf("Computed Remainder:  %s\n", rem);
    char codeword[128];
    strcpy(codeword, data);
    strcat(codeword, rem);
    printf("Transmitted Codeword: %s\n", codeword);
    return 0;
}

```

Experiment 6: 16-Bit Internet Checksum Algorithm in C

Objective: Implement the RFC 1071 One's Complement Internet Checksum algorithm used in IPv4, TCP, and UDP packet headers.

```

#include <stdio.h>
#include <stdint.h>

uint16_t compute_checksum(const uint16_t *buf, int nwords) {
    uint32_t sum = 0;
    for (int i = 0; i < nwords; i++) {
        sum += buf[i];
    }
    // Fold 32-bit sum into 16-bit sum by adding carries
    while (sum >> 16) {
        sum = (sum & 0xFFFF) + (sum >> 16);
    }
    return ~(uint16_t)sum; // One's complement
}

int main() {
    uint16_t packet[] = {0x4500, 0x003c, 0x1c46, 0x4000, 0x4006, 0x0000, 0xac10, 0x0a63, 0xac10, 0x0a0c};
    int nwords = sizeof(packet) / sizeof(packet[0]);
    uint16_t checksum = compute_checksum(packet, nwords);
    printf("Computed 16-Bit Header Checksum: 0x%04X\n", checksum);
    packet[5] = checksum; // Insert checksum into header
    uint16_t verify = compute_checksum(packet, nwords);
    printf("Receiver Verification (Should be 0x0000): 0x%04X %s\n",
        verify, (verify == 0) ? "[VALID]" : "[CORRUPT]");
    return 0;
}

```

Experiment 7: Distance Vector Routing Simulation in C (Bellman-Ford)

Objective: Simulate the Distributed Bellman-Ford Distance Vector Routing protocol in C. Each router maintains a routing table and exchanges distance vectors with immediate neighbors until global convergence.

```
#include <stdio.h>
#define INF 9999
#define NODES 4

int cost[NODES][NODES] = {
    {0, 2, 5, INF},
    {2, 0, 1, 4},
    {5, 1, 0, 1},
    {INF, 4, 1, 0}
};

int dist[NODES][NODES];
int next_hop[NODES][NODES];

void distance_vector() {
    for (int i = 0; i < NODES; i++)
        for (int j = 0; j < NODES; j++) {
            dist[i][j] = cost[i][j];
            next_hop[i][j] = (cost[i][j] < INF && i != j) ? j : -1;
        }

    int updated;
    do {
        updated = 0;
        for (int i = 0; i < NODES; i++) {
            for (int j = 0; j < NODES; j++) {
                for (int k = 0; k < NODES; k++) {
                    if (dist[i][k] + cost[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + cost[k][j];
                        next_hop[i][j] = next_hop[i][k];
                        updated = 1;
                    }
                }
            }
        }
    } while (updated);
}

int main() {
    distance_vector();
    printf("=== Converged Routing Table for Node 0 ===\n");
    printf("Destination\tCost\tNext Hop\n");
    for (int i = 0; i < NODES; i++) {
        printf("Node %d\t\t%d\tNode %d\n", i, dist[0][i], next_hop[0][i]);
    }
    return 0;
}
```

Experiment 8: Link-State Shortest Path Routing (Dijkstra's Algorithm in C)

Objective: Implement Dijkstra's algorithm to compute the shortest path tree and minimum cost routes from a source router to all destination nodes in a network topology.

```

#include <stdio.h>
#include <stdbool.h>
#define V 5
#define INF 9999

int min_distance(int dist[], bool sptSet[]) {
    int min = INF, min_index = -1;
    for (int v = 0; v < V; v++)
        if (!sptSet[v] && dist[v] ≤ min)
            min = dist[v], min_index = v;
    return min_index;
}

void dijkstra(int graph[V][V], int src) {
    int dist[V];
    bool sptSet[V];
    for (int i = 0; i < V; i++) dist[i] = INF, sptSet[i] = false;
    dist[src] = 0;

    for (int count = 0; count < V - 1; count++) {
        int u = min_distance(dist, sptSet);
        sptSet[u] = true;
        for (int v = 0; v < V; v++)
            if (!sptSet[v] && graph[u][v] && dist[u] ≠ INF && dist[u] + graph[u][v] < dist[v])
                dist[v] = dist[u] + graph[u][v];
    }

    printf("Source Router %d → Shortest Paths:\n", src);
    for (int i = 0; i < V; i++) printf("Router %d : Min Cost = %d\n", i, dist[i]);
}

```

Experiment 9: Connection-Oriented TCP Client-Server Socket in C

Objective: Implement a full-duplex TCP iterative echo server and client using BSD socket system calls ('socket', 'bind', 'listen', 'accept', 'connect', 'send', 'recv').

```

/* --- tcp_server.c --- */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, client_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 5);
    printf("🚀 TCP Echo Server listening on port %d...\n", PORT);

    client_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen);
    printf("Client connected: %s\n", inet_ntoa(address.sin_addr));

    int valread = read(client_socket, buffer, BUFFER_SIZE);
    printf("Received from Client: %s\n", buffer);
    send(client_socket, buffer, strlen(buffer), 0); // Echo back
    printf("Echoed response sent.\n");

    close(client_socket);
    close(server_fd);
    return 0;
}

```

Experiment 10: Multi-Client Chat Server using `select()` I/O Multiplexing

Objective: Build a high-concurrency non-blocking TCP chat server using `select()` system call to handle multiple simultaneous client connections without multi-threading race conditions.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/select.h>

#define MAX_CLIENTS 30
#define PORT 8888

int main() {
    int server_fd, new_socket, client_sockets[MAX_CLIENTS] = {0};
    struct sockaddr_in address;
    fd_set readfds;
    char buffer[1024];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 5);
    printf("🐉 Multi-Client Chat Server active on port %d...\n", PORT);

    while (1) {
        FD_ZERO(&readfds);
        FD_SET(server_fd, &readfds);
        int max_sd = server_fd;

        for (int i = 0; i < MAX_CLIENTS; i++) {
            int sd = client_sockets[i];
            if (sd > 0) FD_SET(sd, &readfds);
            if (sd > max_sd) max_sd = sd;
        }

        select(max_sd + 1, &readfds, NULL, NULL, NULL);

        if (FD_ISSET(server_fd, &readfds)) {
            int addrlen = sizeof(address);
            new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen);
            printf("New connection: socket fd %d, ip %s\n", new_socket, inet_ntoa(address.sin_addr));
            for (int i = 0; i < MAX_CLIENTS; i++) {
                if (client_sockets[i] == 0) { client_sockets[i] = new_socket; break; }
            }
        }

        for (int i = 0; i < MAX_CLIENTS; i++) {
            int sd = client_sockets[i];
            if (FD_ISSET(sd, &readfds)) {
                int valread = read(sd, buffer, 1024);
                if (valread == 0) {
                    close(sd);
                    client_sockets[i] = 0;
                } else {
                    buffer[valread] = '\0';
                    // Broadcast message to all connected clients
                    for (int j = 0; j < MAX_CLIENTS; j++) {
                        if (client_sockets[j] > 0 && client_sockets[j] != sd) {
                            send(client_sockets[j], buffer, strlen(buffer), 0);
                        }
                    }
                }
            }
        }
    }
}
```


Experiment 11: Leaky Bucket & Token Bucket Traffic Shaping in C

Objective: Implement the Leaky Bucket algorithm in C to smooth bursty network traffic into a steady, constant-rate packet departure stream.

```
#include <stdio.h>

void leaky_bucket() {
    int bucket_size = 10, leak_rate = 3, n = 5;
    int incoming_packets[] = {4, 8, 2, 7, 3};
    int current_buffer = 0;

    printf("Time\tIncoming\tAccepted\tSent\tRemaining\n");
    for (int t = 0; t < n || current_buffer > 0; t++) {
        int incoming = (t < n) ? incoming_packets[t] : 0;
        int accepted = 0;
        if (incoming + current_buffer ≤ bucket_size) {
            accepted = incoming;
            current_buffer += incoming;
        } else {
            accepted = bucket_size - current_buffer;
            printf("⚠ Drop: %d packets dropped at t=%d\n", incoming - accepted, t);
            current_buffer = bucket_size;
        }

        int sent = (current_buffer ≥ leak_rate) ? leak_rate : current_buffer;
        current_buffer -= sent;
        printf("%d\t%d\t%d\t%d\t%d\t%d\n", t+1, incoming, accepted, sent, current_buffer);
    }
}
```

Experiment 12: Network Subnetting & CIDR Address Calculator in C

Objective: Calculate Network Address, Broadcast Address, Number of Usable Hosts, and Subnet Mask given an IPv4 address and CIDR prefix length.

```
#include <stdio.h>
#include <stdint.h>
#include <arpa/inet.h>

void calculate_subnet(const char *ip_str, int cidr) {
    uint32_t ip;
    inet_pton(AF_INET, ip_str, &ip);
    ip = ntohl(ip);

    uint32_t mask = (cidr == 0) ? 0 : (~0U << (32 - cidr));
    uint32_t net = ip & mask;
    uint32_t bcast = net | ~mask;

    struct in_addr net_addr, bcast_addr, mask_addr;
    net_addr.s_addr = htonl(net);
    bcast_addr.s_addr = htonl(bcast);
    mask_addr.s_addr = htonl(mask);

    printf("IP Address:      %s/%d\n", ip_str, cidr);
    printf("Subnet Mask:    %s\n", inet_ntoa(mask_addr));
    printf("Network ID:     %s\n", inet_ntoa(net_addr));
    printf("Broadcast Address: %s\n", inet_ntoa(bcast_addr));
    printf("Usable Hosts:    %u\n", (cidr ≥ 31) ? 0 : (1U << (32 - cidr)) - 2);
}
```

Comprehensive Viva-Voce Question Bank & Model Answers

Q1. Why does Bit Stuffing insert a '0' after five consecutive '1's?

The HDLC delimiter flag is '01111110' (six consecutive 1s). To prevent a user payload containing six 1s from being falsely interpreted as the end-of-frame flag, the sender injects a '0' after five 1s. The receiver strips this stuffed '0' automatically, preserving frame transparency!

Q2. Differentiate between TCP and UDP Socket Programming.

- **TCP (`SOCK_STREAM`):** Connection-oriented, guarantees in-order delivery, flow control, and congestion control via 3-way handshake (`listen()`, `accept()`, `connect()`).
- **UDP (`SOCK_DGRAM`):** Connectionless, unreliable datagram transmission with zero handshake overhead (`sendto()`, `recvfrom()`). Ideal for real-time video streaming and DNS lookups.

Q3. What is the Count-to-Infinity problem in Distance Vector Routing and how is it resolved?

When a link fails, neighboring nodes continue updating each other with slowly incrementing costs until infinity (16 hops in RIP). Resolved using **Split Horizon** (never send routing info back in the direction it came from) and **Poison Reverse** (advertise failed link cost as ∞).

Q4. How does `select()` enable high-concurrency client-server servers?

Instead of creating a separate thread or process per client ($O(N)$ thread stack memory overhead), `select()` monitors multiple file descriptors simultaneously in a single thread, waking up only when data is ready to be read on a specific socket!

Q5. Explain the significance of the 1's Complement sum in Internet Checksum calculation.

1's complement arithmetic treats end-around carry symmetrically and allows the receiver to sum the payload plus the checksum: if no bits are flipped during transmission, the resulting 16-bit word is identically all 1s (`0xFFFF`), which inverts to `0x0000`.